TECHNICAL DEMONSTRATION REPORT

AI AIDED VISION-TO-FCC ERROR GPS DIVE CORRECTION PROTOTYPE

How an onboard AI camera measures the aircraft's aiming error against a visual target and reports it to the flight controller — so a GPS-guided dive can be corrected with information GPS cannot provide.

July 2026 · Jetson companion computer + SIYI ZR10 gimbal camera + ArduPilot FCC (CubeOrange+)

GPS tells a UAV where *it* is; it cannot tell the UAV where the *target* is once the target is a visual object below. In a GPS-guided dive the aircraft flies to a coordinate — any error in that coordinate, or any target movement, goes uncorrected. This prototype adds the missing feedback: an AI detector locks the target in the camera image, and the system answers one question ten times per second — **"how far is the nose off the target, in degrees of yaw and pitch?"** That pair of numbers is exactly the correction the GPS dive needs, at any dive angle from shallow to near-vertical (0–90°).

1 System setup and communications

Three devices cooperate. The **ZR10 gimbal camera** sees the target and keeps itself pointed at it. The **companion computer** runs the AI detection, steers the gimbal, and does the angle mathematics. The **flight controller (FCC)** supplies the aircraft's attitude and receives the correction over MAVLink.

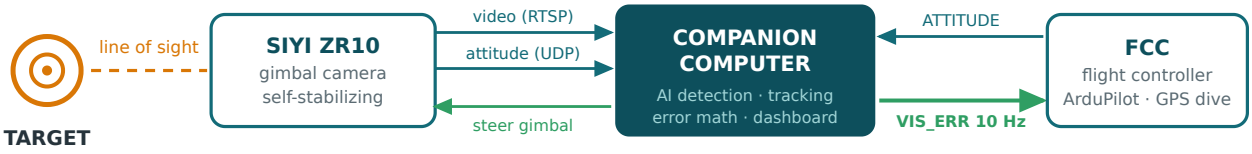

Fig. 1 — Setup and data links. Serial MAVLink carries the aircraft attitude in and the vision correction (`DEBUG_VECT "VIS_ERR"`) out; the FCC forwards telemetry to any ground station.

Link	Transport	Content	Rate
Camera → Companion	RTSP over Ethernet	Live video, 1280×720	stream
Gimbal ↔ Companion	SIYI SDK over UDP	Gimbal attitude in; steering rates out	10 Hz
FCC → Companion	MAVLink, serial	`ATTITUDE`, `HEARTBEAT`	stream
Companion → FCC	MAVLink, serial	`DEBUG_VECT "VIS_ERR"`: x = yaw error, y = pitch error (deg)	10 Hz

A live dashboard shows the camera feed with AI detections (click to lock a target), the correction being streamed, and all three attitude readouts — FCC, gimbal, and the **camera-vs-nose** difference explained next.

2 Two devices, two reference frames — and how they are reconciled

The gimbal is bolted to the airframe just below the FCC, and at power-up both face the same way. It is tempting to conclude their angles should match — **they should not.** The mount only fixes the gimbal's *base*; the camera rides on three motorized axes and moves away from the nose the moment it tracks. And even when both physically point the same way, they *report* their orientation against different references:

Angle	FCC (MAVLink `ATTITUDE`)	Gimbal (SIYI, follow mode)
yaw	**compass heading** — measured from North	**measured from the UAV nose** (left/right of it)
pitch	up/down from the horizon	up/down from the horizon (its own IMU)
roll	tilt from level	tilt from level (its own IMU)

So FCC yaw **+109°** next to gimbal yaw **−28°** is not a contradiction: one says *"the aircraft heads east-southeast"*, the other says *"the camera looks 28° left of the nose"*. They answer different questions and can never be compared or subtracted directly. What both devices **do share is the earth**: each has an IMU referenced to gravity (a common horizon) and yaw can be composed onto a common heading. The earth is the bridge frame:

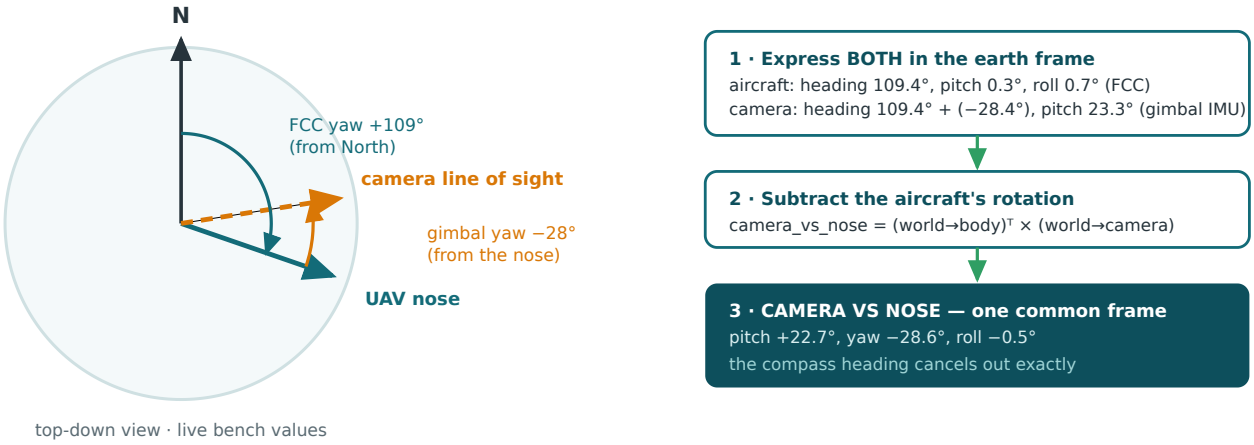

Fig. 2 — Frame reconciliation (`mount_relative_euler()`). Both attitudes are expressed as 3-D rotations in the shared earth frame, then the aircraft's rotation is subtracted; the heading appears in both terms and cancels, leaving pure "where does the camera point relative to the nose".

The camera never "corrects the FCC's angles". It never touches the FCC's attitude at all. The reconciliation happens on the companion computer *before* anything is transmitted, so what reaches the FCC — *"target is 27° left of your nose, 22° above it"* — is already in the aircraft's own body frame, the only frame the FCC thinks in. There is no frame conflict on the wire.

> **Verified on the bench (live hardware, July 7):** with the gimbal centered and both devices aligned, CAMERA VS NOSE reads **−0.15° / −1.30° / −0.68° ≈ 0** — the boresight check. The same geometry evaluated at two different compass headings gives identical output (heading provably cancels), and every dashboard value reproduces through the math to the decimal.

> **The "phantom roll":** that −0.68° roll mirrors the aircraft's own +0.69° roll. The gimbal levels the camera against the *earth*, so seen from a slightly rolled airframe the level camera appears counter-rolled. Real geometry, not an error — and visible live on the dashboard.

3 From a pixel in the image to a correction off the nose

The AI tracker sees the target in the picture, but the camera deliberately points away from where the aircraft points. A target “5° right” *in the image* is therefore not 5° right *of the nose*. Four steps, all in `fcc_landing_feedback.py`, convert one into the other:

1. **Measure the pixel error.** The locked target's box center minus the image center gives an offset in pixels (`VisionOffset`) — e.g. *+776 px right, 69 px up*.
2. **Pixels → camera angles** — `offset_to_angles()`. The lens spreads a known field of view (65° × 39°) across the frame, so `angle = atan(pixels / focal_px)` with `focal_px = (width/2) / tan(FOV/2)`. The tan/atan pair keeps the angle exact even at the frame edges: +776 px → **+27.25°** off the lens centerline.
3. **Find the camera-vs-nose rotation** — `mount_relative_euler()`, the frame reconciliation of Section 2, computed fresh from live gimbal + FCC attitude on every cycle.
4. **Rotate into the aircraft's frame** — `body_frame_angles()`. The target's direction is built as a 3-D ray from step 2, rotated by the camera-vs-nose rotation from step 3, and read out as **yaw error** (deg right of the nose) and **pitch error** (deg above it). This pair is streamed as `DEBUG_VECT "VIS_ERR"`.

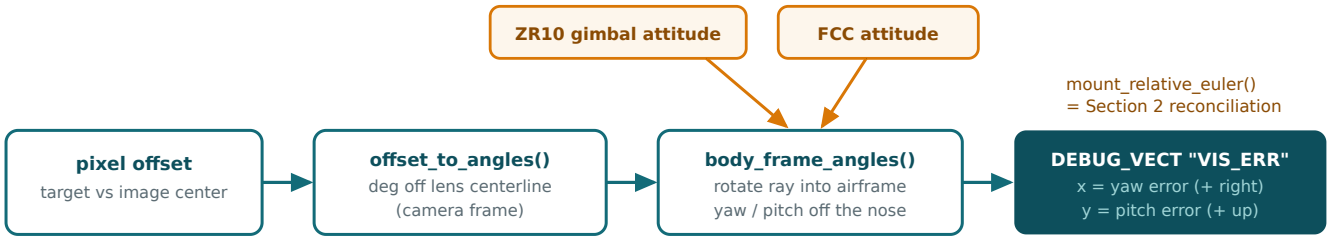

Fig. 3 — The error pipeline. The two attitude sources (amber) are what make the conversion from “in the image” to “off the nose” possible.

Why not just add the angles? Yaw measured by a tilted camera is not the same axis as yaw at the nose, so the code multiplies rotation matrices instead of adding degrees. Live bench proof (July 7, camera pitched 15.5° and yawed 12.4° while tracking):

	naive flat sum	actual (matrices — what was sent)
yaw	12.2° + 27.2° = **+39.5°**	**+40.6°**
pitch	15.6° + 2.6° = **+18.2°**	**+15.8°**

Already 1–2.4° of difference at bench angles. In a steep dive it grows to many degrees: a target 5° right in the image at 60° of camera down-tilt is really about **10°** right of the nose — naive addition would under-steer by half.

4 How the correction aids the GPS dive

The GPS dive gets the aircraft to the right place; the vision correction fixes the last part GPS cannot see. The guidance idea is the one your eyes use on a football: **keep the nose pointed at the target and keep flying — the path collapses onto it.** The two numbers in `VIS_ERR` are exactly that steering instruction, at every instant of the dive:

yaw +3° → steer 3° right · **pitch −2°** → push 2° down · **both ≈ 0** → nose on target, hold the dive

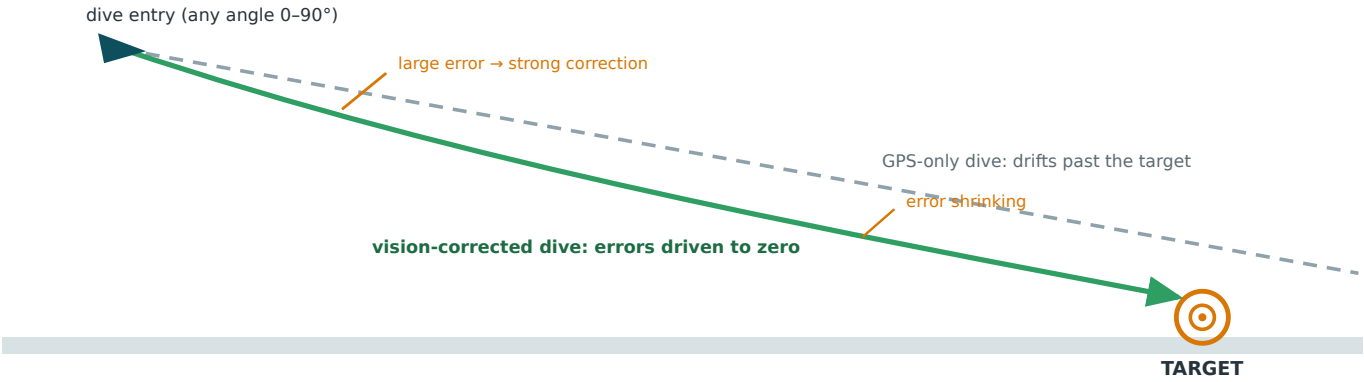

Fig. 4 — Pursuit by error-nulling. Each correction turns the velocity vector a little more onto the target; as the errors shrink the flight path and the line of sight merge. Because the error is measured **off the nose** — not off the vertical — the same math serves a shallow 20° dive and a near-vertical 85° one.

Demonstration scope. In this build the corrections are *reported, not acted on*: `VIS_ERR` is passive telemetry any ground station can plot live. That is the point of the prototype — proving the camera produces a valid, GPS-independent steering signal. Closing the loop is one step: an FCC-side consumer (Lua script or companion commander) reads the same two numbers as rate commands.

Honest limitations. A twisted gimbal mount biases the correction one-to-one — caught on the bench by centering the gimbal and confirming CAMERA VS NOSE ≈ 0 (the dashboard makes this a 10-second check). The gimbal's IMU and the FCC's IMU estimate “level” independently; a fraction-of-a-degree disagreement passes into the correction. Compass error, notably, **cannot** corrupt the output — the heading cancels in the math.

5 Code map

File	Role in one line
`gui_dashboard.py`	Runs everything: five worker threads (video, AI detection, gimbal poll, gimbal steering, FCC/MAVLink) plus the dashboard render loop.
`fcc_landing_feedback.py`	The error mathematics: `offset_to_angles()`, `mount_relative_euler()`, `body_frame_angles()`.
`head_tracker.py`	YuNet CNN detection (the “AI” of the title), click-to-lock matching, and the PID that steers the gimbal.
`siyi_zr10.py`	Minimal SIYI SDK client: gimbal attitude, centering, rate steering over UDP.
`system_math.md`	Plain-language walkthrough of Sections 2–3 with live bench numbers.